

<i>Azure Boards</i>	The Azure Boards service helps to manage projects that use Scrum and Kanban with customisable dashboards and integrated reporting. It helps to track user stories, backlog items, tasks, features and bugs.
<i>Azure Repos</i>	Azure Repos provide Git or Team Foundation Source Control as your Git version. They provide support for pull requests and for branch policies – gated check in.
<i>Pipelines (build pipelines and release pipelines)</i>	Build and release pipelines help in CI/CD implementation using tasks in UI or YAML script that reside in <i>azure-pipelines.yaml</i> . They help to automate different tasks with proper approvals, triggers, and quality gate. You can publish unit test results, code coverage, build quality verification, publish artifacts, etc, using these.
<i>Azure Test Plans</i>	Azure DevOps and Team Foundation Server (TFS) provide support for planned manual testing, user acceptance testing, exploratory testing, and stakeholder feedback for better quality and effective collaboration.
<i>Azure Artifacts</i>	Azure Artifacts service provides features to create and share package feeds for Maven, npm, and NuGet packages. It is preinstalled in Azure DevOps Services, Azure DevOps Server 2019 and 2020, and Team Foundation Server (TFS) 2017 and 2018.
<i>Extensions Marketplace</i>	Visit https://marketplace.visualstudio.com/azuredevops to get more details on the support for extensions in Azure DevOps.

Table 4: Azure DevOps services

The following are some of the key terminologies used in Azure DevOps.

<i>Organisation</i>	We can create one or more organisations in Azure DevOps, and each organisation can have multiple projects
<i>Project</i>	Multiple projects can be used in one organisation using access control. Each project has its own Azure Boards, Azure Repos, Pipelines, and other services

To understand the process, we will try to cover CI/CD for a sample Ionic Cordova application, as it is important to have a hands-on look at Azure DevOps. The steps are given in detail below.

1. Import Ionic Cordova sample application code from GitHub to Azure Repos.
2. Let's do a few configurations in the file *karma.conf.js*
3. Add the required plugins for Junit (unit testing) and Cobertura (code coverage) output:

```
plugins:[
  require('karma-jasmine'),
  require('karma-chrome-launcher'),
  require('karma-mocha-reporter'),
  require('karma-coverage-istanbul-reporter'),
  require('karma-junit-reporter'),
  require('@angular-devkit/build-angular/plugins/
karma')
],
```

4. Add Cobertura in the *reports* key section to *karma.conf.js*:

```
CoverageIstanbulReporter: {
  dir: require('path').join(__dirname, './coverage'),

  reports: ['html', 'lcovonly', 'cobertura'],
  fixWebpackSourcePaths: true
},
reporters: ['progress', 'kjhtml', 'junit'],
```

5. Add *JunitReporter* configuration details in *karma.conf.js* if these are not available in the file:

```
JunitReporter: {
  outputDir: '',
  outputFile: undefined,
  suite: '',
  useBrowserName: true,
  nameFormatter: undefined,
  classNameFormatter: undefined,
  properties:{}
},
```

6. Configure *Headless Browser* so that test cases can be executed:

```
AutoWatch: true,

singleRun: true,
browsers: ['ChromeHeadlessNoSandbox'],
```